

anime-lineart 開発者向けガイド

開発者: Haruki Hoshino

Repository: [anime-lineart](#)

Purpose: アニメイラストを、CLIP STUDIO PAINT で編集可能な SVG 線画へ変換する CLI ツール

1. プロジェクト概要

目的

任意のアニメイラスト画像を、

PNG / JPG / WEBP

↓

SVG 線画

へ変換する。

出力された SVG は：

- CLIP STUDIO PAINT に読み込み可能
- 線を消したり描き足したりできる
- ベクター線として編集可能

を目標とする。

2. 現在の実装状態（MVP）

現在のアルゴリズム：

```
Image
↓
OpenCV bilateral filter
↓
KMeans color quantization
↓
Canny edge detection
↓
Morphological closing
↓
Contour extraction
↓
Path optimization
```

↓
SVG export

特徴：

- OpenCVベース
- CLIのみ
- 1画像ずつ処理
- SVG出力
- 透明背景
- 1px固定線
- 線のみ（fillなし）
- 連続パス優先

3. Repository 構造

```
anime-lineart/  
├── .github/  
│   └── workflows/  
│       └── ci.yml  
  
├── examples/  
│   ├── test.png  
│   ├── test_lineart.svg  
│   ├── sample.png  
│   └── sample_lineart.svg  
  
├── src/  
│   ├── anime_lineart/  
│   │   ├── cli/  
│   │   │   └── main.py  
│   │   ├── config/  
│   │   │   └── config.py  
│   │   ├── core/  
│   │   │   ├── segmentation/  
│   │   │   ├── contour/  
│   │   │   ├── optimization/  
│   │   │   └── pipeline/  
│   │   ├── exporters/  
│   │   ├── models/  
│   │   └── utils/  
└── tests/  
    └── test_cli.py
```

```
├─ pyproject.toml
├─ uv.lock
└─ README.md
```

4. 開発環境

Python

Python 3.11+

現在：

3.12.x

パッケージ管理

使用：

uv

理由：

- 高速
- modern Python ecosystem
- lock file管理が簡単

5. 開発時の基本コマンド

仮想環境込みで実行

CLI実行

```
uv run anime-linearart examples/sample.png
```

出力：

```
examples/sample_lineart.svg
```

Python 実行

普通の：

```
python script.py
```

は禁止。

必ず：

```
uv run python script.py
```

を使う。

理由：

uv 環境内の dependency を使うため。

6. 品質チェック

コード変更後は必ず実行。

Ruff

```
uv run ruff check .
```

整形：

```
uv run ruff format .
```

Mypy

型チェック：

```
uv run mypy src
```

成功例：

```
Success: no issues found
```

Pytest

テスト：

```
uv run pytest
```

成功例：

```
1 passed
```

7. Git 管理ルール

基本方針

ブランチ：

```
main only
```

理由：

小規模プロジェクトのため。

開発フロー

1. 実装

コードを書く。

2. 品質チェック

必ず実行：

```
uv run ruff check .  
uv run mypy src  
uv run pytest
```

3. Git状態確認

```
git status
```

4. commit

全部追加：

```
git add .
```

コミット：

```
git commit -m "Meaningful message"
```

例：

```
git commit -m "Improve contour extraction"
```

悪い例：

```
git commit -m "fix"
```

5. push

```
git push
```

GitHub確認

repo：

```
https://github.com/hhhcord/anime-linear
```

8. GitHub Actions

pushすると自動実行。

内容：

```
ruff  
mypy  
pytest
```

失敗すると：

 赤

成功：

 緑

Actions タブで確認。

9. 重要ルール

__pycache__ は commit しない

.gitignore 管理済み。

もし混入したら：

```
git rm -r --cached src/**/__pycache__
```

dependency追加

追加時：

例：

```
uv add pillow
```

開発専用：

```
uv add --dev black
```

自動で：

```
pyproject.toml  
uv.lock
```

が更新される。

必ず commit。

import ルール

絶対 import を使う。

OK：

```
from anime_lineart.config import LineArtConfig
```

NG：

```
from ..config import LineArtConfig
```

理由：

src layout と相性が悪い。

10. アーキテクチャ思想

責務分離。

config/

設定管理。

例：

```
LineArtConfig
```

segmentation/

色領域分割。

現在：

```
OpenCVSegmenter
```

将来：

```
AISegmenter
```

追加予定。

contour/

輪郭抽出。

現在：

```
Canny + contour
```

optimization/

SVG向け最適化。

目的：

- path smoothing
 - contour simplification
 - path continuity
-

pipeline/

処理統合。

入口：

AnimeLineartPipeline

ここが：

image → svg

全体フローを管理。

exporter/

出力管理。

現在：

SVGExporter

将来：

PSDExporter

追加可能。

11. 将来ロードマップ

Phase 1（現在）

MVP。

OpenCV → SVG

Phase 2

品質改善。

目標：

- 顔線保持
- 髪線保持

- 主要パーツ優先
- ノイズ除去

予定：

```
region boundary extraction
```

Phase 3

アニメ特化。

導入候補：

```
k-means segmentation  
superpixel  
adaptive threshold
```

Phase 4

AI化。

候補：

```
U2Net  
ControlNet  
Line extraction model
```

12. よく使うコマンドまとめ

実行

```
uv run anime-linearart examples/sample.png
```

テスト

```
uv run pytest
```

型チェック

```
uv run mypy src
```

lint

```
uv run ruff check .
```

format

```
uv run ruff format .
```

commit

```
git add .  
git commit -m "message"  
git push
```

13. 開発原則

優先順位：

```
品質 > 安定性 > 速度
```

対象：

```
1000~3000px  
アニメイラスト  
透明背景 or 単色背景
```

出力：

CLIP STUDIO PAINT編集可能SVG

目的：

「描いたり消したりできる線画」